Focus Group

Can LLM code ethically as a developer would do?

Idea

The Context

When developers write code, they make decisions about concerns that are not fully determined by functional requirements.

The Impact

These decisions are value-laden as they can shape ethics-relevant properties of the resulting code.

What if the code is written by an LLM?

01

Step 1

Validate Soft Ethics Schema

02

Step 2

Validate How the Profile Is Elicited

Focus Group Outline

01

Step 1 - Objective

Validate the conceptual adequacy, coverage, and distinctiveness of the proposed ethical aspects.

02

Step 2 - Objective

Identify suitable quantitative elicitation mechanisms for translating developers' ethical preferences into aspect-level scores.

Focus Group Outline

01

Step 1 - Objective

Validate the conceptual adequacy, coverage, and distinctiveness of the proposed ethical aspects.

02

Step 2 - Objective

Identify suitable quantitative elicitation mechanisms for translating developers' ethical preferences into aspect-level scores.

Dimension	Aspect	Description	Rationale
Equity	*Fairness*	Developers may seek to avoid artifacts that encode stereotypes, exclusionary categories, unjustified sensitive data collection, or differential treatment without a clear functional reason.	Code can embed assumptions that affect users directly, even when the code is syntactically correct. For example, a form may restrict gender to binary options, require unnecessary demographic attributes, use culturally narrow examples, or validate names and addresses in ways that exclude some users.
	Inclusiveness	Developers may prioritize artifacts that can be used by people with different abilities, languages, devices, contexts, and needs, and that avoid implicitly centering only a narrow group of users.	Code may assume a limited class of users, interaction styles, devices, or linguistic and cultural backgrounds. This preference can be reflected in accessible component structure, keyboard navigation, clear labels, flexible inputs, language-aware behavior, and design choices that support broader participation and representation.
Accountability	*Transparency and Explainability*	Developers may prefer code artifacts whose behavior is understandable to both maintainers and end users. This includes readable implementation logic as well as user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what happens after submission. At the artifact level, this preference concerns both developer-to-developer understanding, e.g., clear structure and meaningful comments, and developer-to-user understanding, e.g., explanations, labels, and feedback.
	Traceability	Developers may prefer artifacts that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. At the artifact level, this can be reflected in explicit submission states, meaningful error handling, traceable validation rules, audit-oriented events, and mechanisms for users or maintainers to review actions.
Sustainability	*Energy Consumption*	Developers may prefer artifacts that avoid unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while introducing avoidable inefficiencies, such as heavy libraries, redundant computations, excessive client-side updates, or unnecessary requests. At the artifact level, this preference can be observed in implementation choices that influence resource use.

Do these aspects represent a meaningful ethical consideration in software development?

Dimension	Aspect	Description	Rationale
Equity	*Fairness*	Developers may seek to avoid artifacts that encode stereotypes, exclusionary categories, unjustified sensitive data collection, or differential treatment without a clear functional reason.	Code can embed assumptions that affect users directly, even when the code is syntactically correct. For example, a form may restrict gender to binary options, require unnecessary demographic attributes, use culturally narrow examples, or validate names and addresses in ways that exclude some users.
	Inclusiveness	Developers may prioritize artifacts that can be used by people with different abilities, languages, devices, contexts, and needs, and that avoid implicitly centering only a narrow group of users.	Code may assume a limited class of users, interaction styles, devices, or linguistic and cultural backgrounds. This preference can be reflected in accessible component structure, keyboard navigation, clear labels, flexible inputs, language-aware behavior, and design choices that support broader participation and representation.
Accountability	*Transparency and Explainability*	Developers may prefer code artifacts whose behavior is understandable to both maintainers and end users. This includes readable implementation logic as well as user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what happens after submission. At the artifact level, this preference concerns both developer-to-developer understanding, e.g., clear structure and meaningful comments, and developer-to-user understanding, e.g., explanations, labels, and feedback.
	Traceability	Developers may prefer artifacts that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. At the artifact level, this can be reflected in explicit submission states, meaningful error handling, traceable validation rules, audit-oriented events, and mechanisms for users or maintainers to review actions.
Sustainability	*Energy Consumption*	Developers may prefer artifacts that avoid unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while introducing avoidable inefficiencies, such as heavy libraries, redundant computations, excessive client-side updates, or unnecessary requests. At the artifact level, this preference can be observed in implementation choices that influence resource use.

Are the definitions sufficiently clear?

Dimension	Aspect	Description	Rationale
Equity	*Fairness*	Developers may seek to avoid artifacts that encode stereotypes, exclusionary categories, unjustified sensitive data collection, or differential treatment without a clear functional reason.	Code can embed assumptions that affect users directly, even when the code is syntactically correct. For example, a form may restrict gender to binary options, require unnecessary demographic attributes, use culturally narrow examples, or validate names and addresses in ways that exclude some users.
	Inclusiveness	Developers may prioritize artifacts that can be used by people with different abilities, languages, devices, contexts, and needs, and that avoid implicitly centering only a narrow group of users.	Code may assume a limited class of users, interaction styles, devices, or linguistic and cultural backgrounds. This preference can be reflected in accessible component structure, keyboard navigation, clear labels, flexible inputs, language-aware behavior, and design choices that support broader participation and representation.
Accountability	*Transparency and Explainability*	Developers may prefer code artifacts whose behavior is understandable to both maintainers and end users. This includes readable implementation logic as well as user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what happens after submission. At the artifact level, this preference concerns both developer-to-developer understanding, e.g., clear structure and meaningful comments, and developer-to-user understanding, e.g., explanations, labels, and feedback.
	Traceability	Developers may prefer artifacts that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. At the artifact level, this can be reflected in explicit submission states, meaningful error handling, traceable validation rules, audit-oriented events, and mechanisms for users or maintainers to review actions.
Sustainability	*Energy Consumption*	Developers may prefer artifacts that avoid unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while introducing avoidable inefficiencies, such as heavy libraries, redundant computations, excessive client-side updates, or unnecessary requests. At the artifact level, this preference can be observed in implementation choices that influence resource use.

Are the aspects clearly distinct from the others?

Dimension	Aspect	Description	Rationale
Equity	*Fairness*	Developers may seek to avoid artifacts that encode stereotypes, exclusionary categories, unjustified sensitive data collection, or differential treatment without a clear functional reason.	Code can embed assumptions that affect users directly, even when the code is syntactically correct. For example, a form may restrict gender to binary options, require unnecessary demographic attributes, use culturally narrow examples, or validate names and addresses in ways that exclude some users.
	Inclusiveness	Developers may prioritize artifacts that can be used by people with different abilities, languages, devices, contexts, and needs, and that avoid implicitly centering only a narrow group of users.	Code may assume a limited class of users, interaction styles, devices, or linguistic and cultural backgrounds. This preference can be reflected in accessible component structure, keyboard navigation, clear labels, flexible inputs, language-aware behavior, and design choices that support broader participation and representation.
Accountability	*Transparency and Explainability*	Developers may prefer code artifacts whose behavior is understandable to both maintainers and end users. This includes readable implementation logic as well as user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what happens after submission. At the artifact level, this preference concerns both developer-to-developer understanding, e.g., clear structure and meaningful comments, and developer-to-user understanding, e.g., explanations, labels, and feedback.
	Traceability	Developers may prefer artifacts that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. At the artifact level, this can be reflected in explicit submission states, meaningful error handling, traceable validation rules, audit-oriented events, and mechanisms for users or maintainers to review actions.
Sustainability	*Energy Consumption*	Developers may prefer artifacts that avoid unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while introducing avoidable inefficiencies, such as heavy libraries, redundant computations, excessive client-side updates, or unnecessary requests. At the artifact level, this preference can be observed in implementation choices that influence resource use.

Are there relevant aspects that are missing?

Focus Group Outline

01

Step 1 - Objective

Validate the conceptual adequacy, coverage, and distinctiveness of the proposed ethical aspects.

02

Step 2 - Objective

Identify suitable quantitative elicitation mechanisms for translating developers' ethical preferences into aspect-level scores.

Proposed profile elicitation mechanism

We derived 4 to 5 statement for each ethical aspect:

When you write the code, to what extent do you agree with the following statements?

Questions:

- Is agreement with concrete preference statements an appropriate way to quantify a developer's preference for the corresponding ethical aspect?
- Is a five-point Likert scale suitable for capturing meaningful differences between developers?

Statements

Please read the statements *[5 minutes]*.

<statements in the initial_statements.pdf file>

Questions:

- Do the statements represent a meaningful aspect of the intended aspect?
- Are some important aspects missing?